Timeline 0

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

Can you find the flag in this disk image? Wrap what you find in the picoCTF flag format.

💡 Hints

- Create a Sleuthkit MAC timeline!
- Sloppy timestomping can yield strange (very old) timestamps

---

🧰 Tools Used

- `7z` – Extract compressed files
- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `fdisk` – View partition tables
- `fls` – List files in a disk image (Sleuth Kit)
- `mactime` – Create a MAC timeline (Sleuth Kit)
- `icat` – Retrieve a file by inode number (Sleuth Kit)
- `grep` – Filter output
- `base64` – Decode Base64-encoded data

🔍 Complete Solution

Step 1: Download and Extract

Download the disk image named `partition4.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x partition4.img.gz
```

This decompresses the gzipped file and creates the raw disk image `partition4.img` .

Step 2: Basic File Inspection

Verify the file type and understand its structure:

```
file partition4.img
```

Expected output:

```
partition4.img: Linux rev 1.0 ext4 filesystem data, UUID=7a00e9da-98f8-4f0f-b257-95edf422d902 (extents) (64bit) (large files) (huge files)
```

The output confirms this is an **ext4 filesystem** (Linux).

Step 3: Search for Visible Text

Disk images often contain strings embedded directly in the data. Extract all readable character sequences and search for the flag:

```
strings partition4.img | grep -i "picoCTF"
```

Output: (no results)

No flag appears yet. We need to dig deeper using forensic timeline analysis.

Step 4: Inspect Metadata

Examine the disk image metadata:

```
exiftool partition4.img
```

Expected output:

```
ExifTool Version Number         : 13.50
File Name                       : partition4.img
Directory                       : .
File Size                       : 490 MB
File Modification Date/Time     : 2025:12:01 22:53:52+02:00
File Access Date/Time           : 2026:05:13 17:52:57+02:00
File Inode Change Date/Time     : 2026:05:13 17:52:52+02:00
File Permissions                : -rw-r--r--
Error                           : First 1024 bytes of file is binary zeros
```

Nothing suspicious here.

🔬 Sleuth Kit Timeline Analysis

The hints suggest creating a **MAC timeline** using The Sleuth Kit.

Step 5: Extract File List with fls

```
fls -m / -r partition4.img > output.txt
```

Option	Description
`-m /`	Output in body file format for mactime
`-r`	Recursively list all contents

This command generates a **body file** containing metadata for every file and directory in the disk image.

Step 6: Create Chronological Timeline with mactime

```
mactime -b output.txt > timeline.txt
```

This creates `timeline.txt` sorted by timestamp (from oldest to newest).

Step 7: Understand MACB Timestamps

The `mactime` output includes a column with letters **m a c b** representing the four timestamps tracked by Linux filesystems:

Letter	Meaning	Description
m	Modify	Data modification time
a	Access	File access (read) time
c	Change	Metadata change time (permissions, ownership)
b	Birth	File creation time

> 💡 **Anti-forensic indicator (Timestomping)** : When a tool intentionally modifies timestamps to hide activity, all four flags (`macb`) may appear together, often overwritten with a suspicious timestamp (like a very old date).

The hint mentions **"sloppy timestomping can yield strange (very old) timestamps"** – this is our key clue!

Step 8: Filter Suspicious macb Entries

Look for files with all four `macb` flags and unusually old timestamps:

```
grep "macb" timeline.txt | head -n 20
```

Output (excerpt):

```
Tue Jan 01 1985 19:00:00       41 macb r/rrw-r--r-- 0        0        4945
/bin/bcab
                            45472 m... r/rrw-r--r-- 0        0        2339
/lib/modules/6.6.116-0-lts/kernel/drivers/net/ethernet/cadence/macb.ko.gz
```

```
Tue Nov 11 2025 22:01:17    12288 macb d/drwx------ 0        0        11
/lost+found
                             1024 macb d/drwxr-xr-x 0        0        13        /proc
                                0 macb r/rrw------- 0        0        19
/lib/apk/db/lock
                             1024 macb d/drwxr-xr-x 0        0        32399
/etc/apk/protected_paths.d
                             1024 macb d/drwxr-xr-x 0        0        32414
/etc/doas.d
                             1024 macb d/drwxr-xr-x 0        0        64769     /boot
                             1024 macb d/drwxr-xr-x 0        0        64770     /home
                             1024 macb d/drwxr-xr-x 0        0        64772     /dev
```

> **Key finding**: The file `/bin/bcab` has a timestamp from **1985** (extremely old) and all four `macb` flags. This is a clear sign of **timestomping** (intentional timestamp manipulation)!

The `41` at the beginning of the line is the inode number (4945).

Step 9: Extract the File by Inode

Use `icat` to retrieve the file directly by its inode number without mounting the filesystem:

```
icat partition4.img 4945
```

Output:

```
NzFtMzExbjNfMHU3MTEzcl9oM3JfNDNhMmU3YWYK
```

This appears to be a **Base64-encoded** string.

Step 10: Decode the Base64

```
echo 'NzFtMzExbjNfMHU3MTEzcl9oM3JfNDNhMmU3YWYK' | base64 -d
```

Output:

```
<REDACTED>
```

This reveals the flag!

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, decoding the Base64 string will reveal the complete flag.

🧠 Key Concepts

- **Raw Disk Image**: A sector-by-sector copy of a storage device that can contain a filesystem directly.
- **ext4 Filesystem**: The native Linux filesystem used in this image.
- **MAC Timestamps**: Modify, Access, Change, Birth – four timestamps tracked by Linux filesystems.
- **Timestomping**: An anti-forensic technique that intentionally modifies file timestamps to hide malicious activity.
- **Sloppy Timestomping**: Poorly executed timestamp manipulation that leaves obvious traces (e.g., very old dates like 1985).
- **macb Indicator**: When all four timestamps appear together or are identical, it may indicate tampering.
- **Sleuth Kit (TSK)**: A collection of command-line tools for forensic analysis of disk images.
- **fls**: Lists files and directories in a disk image, outputting metadata in body file format.
- **mactime**: Creates a chronological timeline from body files.
- **icat**: Retrieves a file by its inode number without mounting the filesystem.
- **Inode**: A unique identifier for a file or directory in Unix/Linux filesystems.